

Multimodal Text-to-Image Application

Multimodal Text-to-Image Application

1. Concept Introduction
 - 1.1 What is Text-to-Image?
 - Core Principles
 - 1.2 What is FastSDCPU?
 - Core Features
 - Applicable Scenarios
2. Project Deployment
 - 2.1 Deployment Environment
 - 2.2 LAN Access
 - 2.3 Using the Text-to-Image Function
 - 2.4 Startup Method After Successful Deployment

Since Ollama does not support text-to-image functionality, we need to use other tools to implement local text-to-image generation. Currently, voice control for text-to-image is not supported, so the content of this article is the same as the offline text multimodal text-to-image application.

1. Concept Introduction

1.1 What is Text-to-Image?

Text-to-Image is an artificial intelligence technology that automatically generates corresponding images from **text descriptions**. Simply input a piece of text (e.g., "a Shiba Inu wearing sunglasses surfing on the beach"), and the AI model will generate an image that matches the description based on semantic understanding, without any need for drawing or design skills.

Core Principles

1. **Language Understanding:** The AI first parses your text, identifying keywords (such as "Shiba Inu," "sunglasses," "beach").
2. **Image Generation:** Based on training with massive amounts of image data, the AI transforms the text into visual elements and combines them into a reasonable picture.
3. **Style Adaptation:** You can specify a style (realistic, cartoon, watercolor, etc.), and the AI will adjust the generation effect.

1.2 What is FastSDCPU?

FastSDCPU is an open-source **Stable Diffusion text-to-image** project optimized for **CPU devices**. Through algorithmic and engineering optimizations, it enables the rapid generation of high-quality images on ordinary computers without a GPU, significantly lowering the hardware threshold for AI painting.

Core Features

1. **Pure CPU Operation:** No need for a dedicated graphics card; it can run directly on laptops, desktops, or edge devices.
2. **High-speed Inference:** Combined with technologies like Latent Consistency Models (LCM), it only takes 4–8 steps to generate an image, greatly reducing waiting time.
3. **Lightweight Deployment:** Supports model quantization (such as INT8) and OpenVINO acceleration, reducing resource consumption and improving response efficiency.

Applicable Scenarios

- Localized AI creation tools
- Image generation services on corporate intranets
- Teaching demonstrations and prototype development
- Lightweight deployment in resource-constrained environments

2. Project Deployment

2.1 Deployment Environment

Note: If you are using our factory image, there is no need to deploy the environment; you can skip the deployment steps. Refer directly to <2.4 Startup Method After Successful Deployment> below to start. Then, return to <2.3 Using the Text-to-Image Function> to learn how to use it.

Open a terminal and execute the following code:

```
# If git is not on the motherboard, run this first
sudo apt update
sudo apt install git -y
sudo apt install python3.10-venv -y

# Add environment variables
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc

# Clone the project code
git clone https://github.com/rupesths/fastsdcpu.git
cd fastsdcpu

# Create a virtual environment and install dependencies
python -m venv venv
source venv/bin/activate

# Install uv
curl -Ls https://astral.sh/uv/install.sh | sh (This step may fail in China
without a proxy. If it doesn't exceed, you can skip this step and execute the
next command)

# If the curl command to install uv in the previous step fails, run these three
commands
wget https://mirrors.huaweicloud.com/astral/uv/0.8.4/uv-aarch64-unknown-linux-gnu
-O ~/.local/bin/uv
```

```
chmod +x ~/.local/bin/uv
~/.local/bin/uv --version

chmod +x install.sh start-webui.sh
./install.sh --disable-gui
```

Installation successful, press any key to exit:

```
+ tomesd==0.1.3
+ toml==2.2.1
+ tomlkit==0.12.0
- torch==2.7.1+cpu
+ torch==2.5.1
+ torchvision==0.20.1
+ tqdm==4.67.1
+ transformers==4.48.0
+ typer==0.16.0
- typing-extensions==4.12.2
+ typing-extensions==4.8.0
+ tzdata==2025.2
+ urllib3==2.5.0
+ uvicorn==0.35.0
+ websockets==12.0
+ xxhash==3.5.0
+ yarl==1.20.1
+ zipp==3.23.0
FastSD CPU installation completed,press any key to continue...
```

2.2 LAN Access

Before starting, you need to modify a file to support LAN access; otherwise, the web UI can only be accessed locally:

```
vim ~/.fastsdcpu/src/frontend/webui/ui.py
```

After opening the ui.py file, go to the last line and find the line **webui.launch(share=share)**, change it to **webui.launch(server_name="0.0.0.0",share=share)**

Then save.

Start:

```
./start-webui.sh
```

```

Found 7 stable diffusion models in config/stable-diffusion-models.txt
Found 4 LCM-LoRA models in config/lcm-lora-models.txt
Found 10 OpenVINO LCM models in config/openvino-lcm-models.txt
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/controlnet_aux/segment_anything/modeling/tiny_vit_sam.py:654:
UserWarning: Overwriting tiny_vit_5m_224 in registry with controlnet_aux.segment_anything.modeling.tiny_vit_sam.tiny_vit_5m_224. This is because the name being registered conflicts with an existing name. Please check if this is not expected.
    return register_model(fn_wrapper)
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/controlnet_aux/segment_anything/modeling/tiny_vit_sam.py:654:
UserWarning: Overwriting tiny_vit_11m_224 in registry with controlnet_aux.segment_anything.modeling.tiny_vit_sam.tiny_vit_11m_224. This is because the name being registered conflicts with an existing name. Please check if this is not expected.
    return register_model(fn_wrapper)
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/controlnet_aux/segment_anything/modeling/tiny_vit_sam.py:654:
UserWarning: Overwriting tiny_vit_21m_224 in registry with controlnet_aux.segment_anything.modeling.tiny_vit_sam.tiny_vit_21m_224. This is because the name being registered conflicts with an existing name. Please check if this is not expected.
    return register_model(fn_wrapper)
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/controlnet_aux/segment_anything/modeling/tiny_vit_sam.py:654:
UserWarning: Overwriting tiny_vit_21m_384 in registry with controlnet_aux.segment_anything.modeling.tiny_vit_sam.tiny_vit_21m_384. This is because the name being registered conflicts with an existing name. Please check if this is not expected.
    return register_model(fn_wrapper)
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/controlnet_aux/segment_anything/modeling/tiny_vit_sam.py:654:
UserWarning: Overwriting tiny_vit_21m_512 in registry with controlnet_aux.segment_anything.modeling.tiny_vit_sam.tiny_vit_21m_512. This is because the name being registered conflicts with an existing name. Please check if this is not expected.
    return register_model(fn_wrapper)
Starting web UI mode
No lora models found, please add lora models to /home/sunrise/fastsdcpu/lora_models directory
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/gradio/components/dropdown.py:226: UserWarning: The value passed into gr.Dropdown() is not in the list of choices. Please update the list of choices to include: or set allow_custom_value=True.
    warnings.warn(
/home/sunrise/fastsdcpu/env/lib/python3.11/site-packages/gradio/components/base.py:194: UserWarning: show_label has no effect when container is False.
    warnings.warn("show_label has no effect when container is False.")
* Running on local URL:  http://0.0.0.0:7860

To create a public link, set `share=True` in `launch()`.

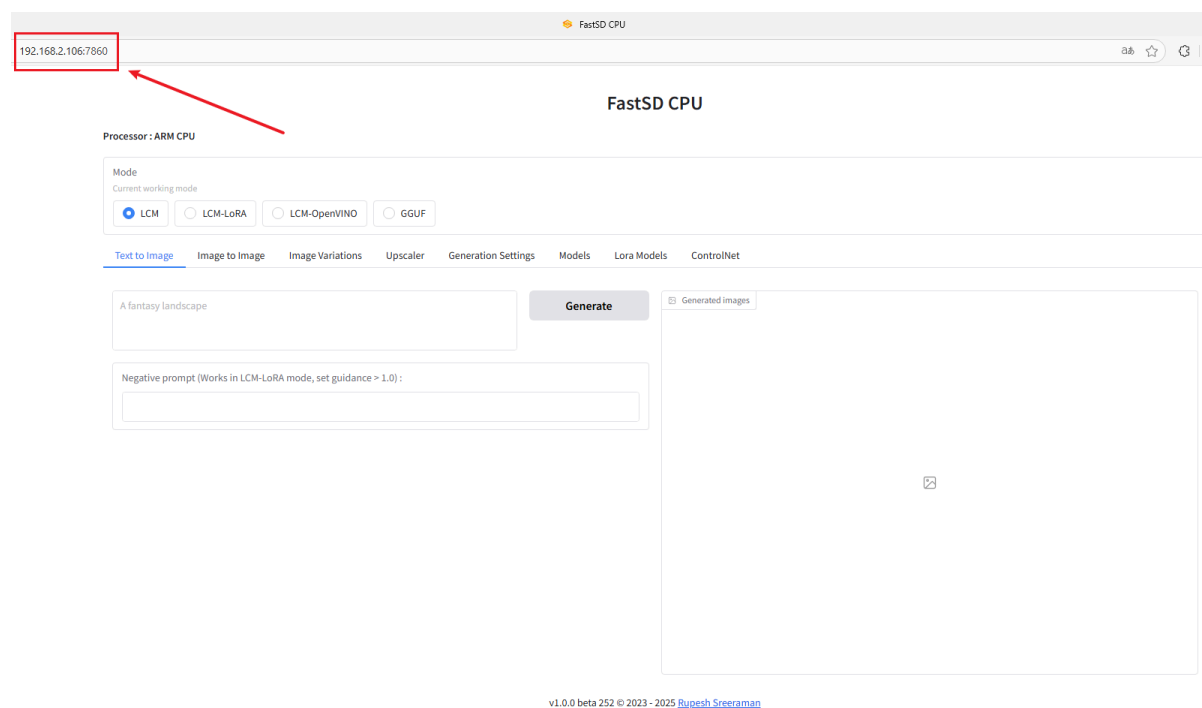
```

Then you can enter your motherboard's IP:7860 in the browser to access this web UI.

2.3 Using the Text-to-Image Function

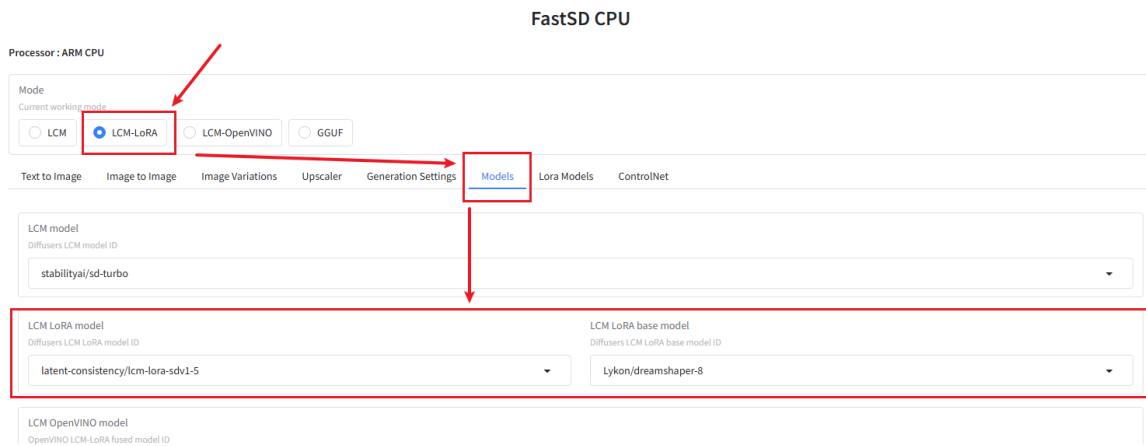
Use ifconfig in the terminal to query our motherboard's IP, for example, mine is 192.168.2.106.

Then we open the browser and enter **your motherboard's IP:7860**. For example, I would enter 192.168.2.106:7860, and then I can enter the web UI.

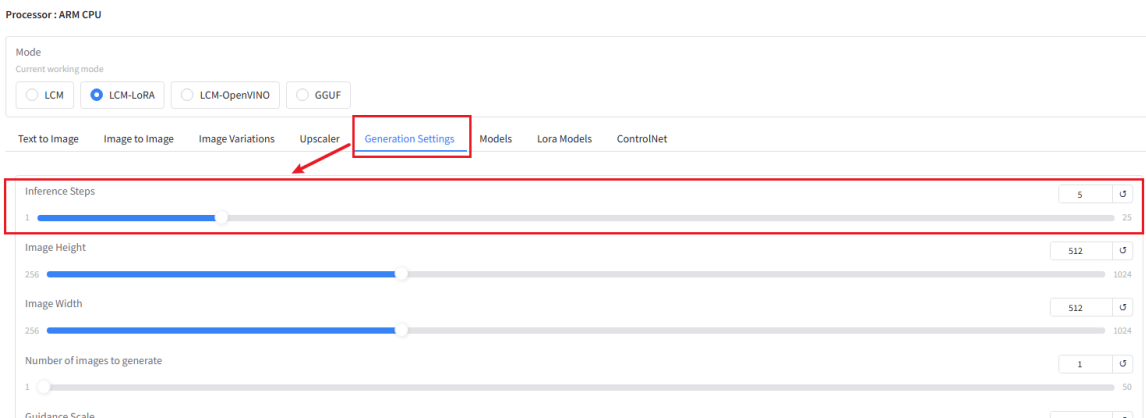


Next, we click on LCM-LoRA; this model has a smaller memory footprint. If you want to use other models, you can research them yourself.

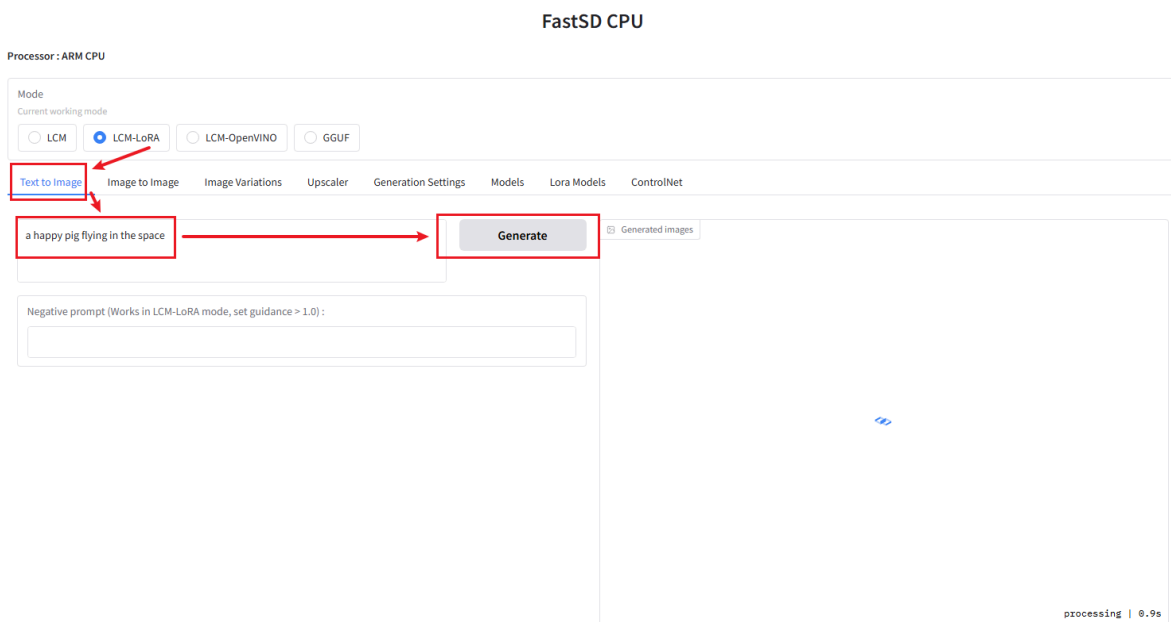
Then click on Models, you can see the model settings for LCM LoRA. You can change to the model you want, or you can choose the default one like me.



Next, click on Generation Settings and increase the Inference Steps to improve the quality of the generated images. I'll set it to 5 here.



Then we go back to our Text to Image, enter what we want to generate in the dialog box, and then press Generate to start generating the image.



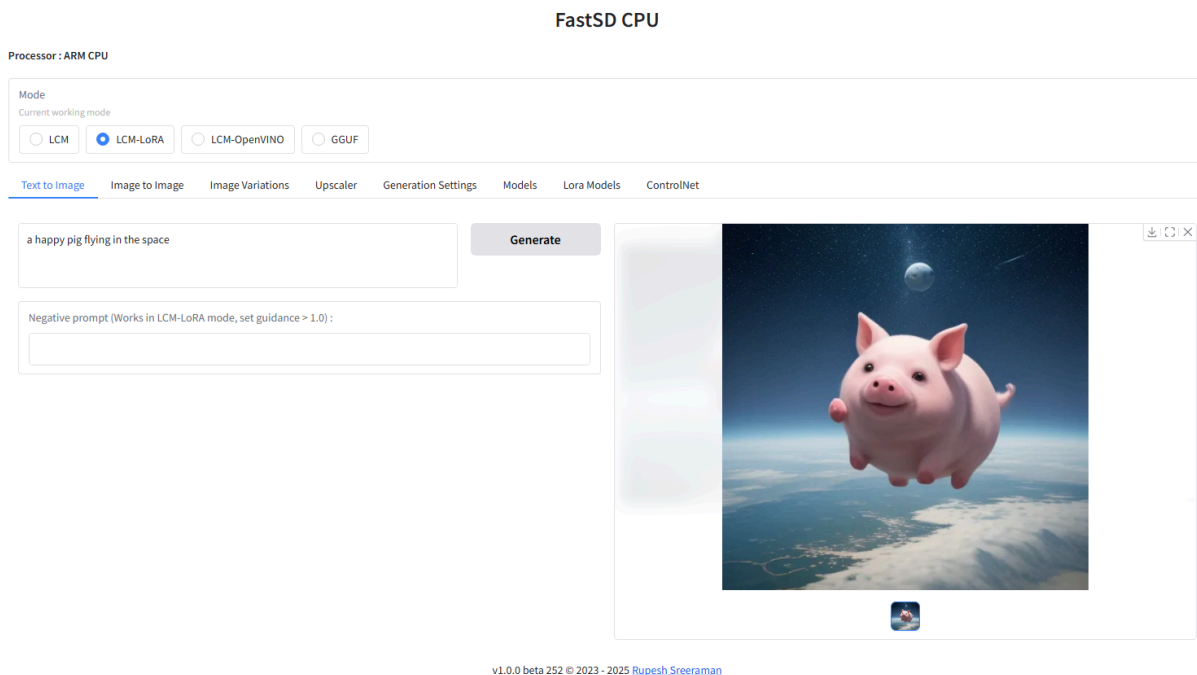
If this is your first time using it, you need to download the model. You can see in the terminal that the model selected by default is being downloaded. After it is downloaded, the text-to-image function will be executed.

```

'init_image': None,
'pcm_lora': {'base_model_id': 'Lykon/dreamshaper-8',
             'pcm_lora_id': 'latent-consistency/lcm-lora-sdv1-5'},
'pcm_model_id': 'stabilityai/sd-turbo',
'lora': {'enabled': False,
         'fuse': True,
         'models_dir': '/home/sunrise/fastsdcpu/lora_models',
         'path': '',
         'weight': 0.5},
'negative_prompt': '',
'number_of_images': 1,
'openvino_pcm_model_id': 'rupeshs/sd-turbo-openvino',
'prompt': 'a happy pig flying in the space',
'rebuild_pipeline': False,
'seed': 123123,
'strength': 0.6,
'token_merging': 0.0,
'use_gguf_model': False,
'use_pcm_lora': True,
'use_offline_model': False,
'use_openvino': False,
'use_safety_checker': False,
'use_seed': False,
'use_tiny_auto_encoder': False}
***** Init LCM-LoRA pipeline - Lykon/dreamshaper-8 *****
model_index.json: 100%| 642/642 [00:00<00:00, 2.80MB/s]
config.json: 100%| 796/796 [00:00<00:00, 3.81MB/s]
config.json: 100%| 724/724 [00:00<00:00, 3.74MB/s]
special_tokens_map.json: 100%| 472/472 [00:00<00:00, 2.27MB/s]
merges.txt: 525kB [00:00, 6.20MB/s] | 0.00/472 [00:00<?, ?B/s]
preprocessor_config.json: 100%| 520/520 [00:00<00:00, 2.53MB/s]
tokenizer_config.json: 100%| 737/737 [00:00<00:00, 3.54MB/s]
config.json: 1.87kB [00:00, 5.15MB/s] | 0.00/1.22G [00:00<?, ?B/s]
vocab.json: 1.06MB [00:00, 9.26MB/s] | 0.00/520 [00:00<?, ?B/s]
scheduler_config.json: 100%| 614/614 [00:00<00:00, 2.99MB/s]
config.json: 100%| 756/756 [00:00<00:00, 3.99MB/s]
model.safetensors: 2%| 10.5M/492M [00:00<00:23, 20.3MB/s]
model.safetensors: 32%| 157M/492M [00:36<01:24, 3.98MB/s]
model.safetensors: 12%| 147M/1.22G [00:34<04:28, 3.98MB/s]
diffusion_pytorch_model.safetensors: 41%| 136M/335M [00:34<00:50, 3.96MB/s]
diffusion_pytorch_model.safetensors: 4%| 136M/3.44G [00:34<13:52, 3.97MB/s]

```

Generated result:



It has been tested that this project has better support for English, and the generated images will be more in line with the text. It is recommended to use English descriptions to generate images.

2.4 Startup Method After Successful Deployment

```

cd fastsdcpu # Enter the fastsdcpu directory
source venv/bin/activate # Enter the virtual environment
./start-webui.sh # Start the webui

```

After the webui starts successfully, enter your motherboard's IP:7860 in the browser to start the text-to-image function.